

---

**title: “Práctico 1 — Resuelto paso a paso” subtitle: “Detección y corrección de errores · Representación Interna de Datos — Arquitectura de Computadoras FING UdelaR”**

# Práctico 1 — Resuelto paso a paso

**Nota importante sobre este documento:** cada ejercicio se resolvió de manera **independiente**, verificando cada resultado numérico contra el enunciado (sumando, multiplicando o reconstruyendo el valor original) antes de darlo por bueno. Donde el material de referencia del curso ( [Ejercicios Resueltos.pdf](#) ) tenía un valor que no coincidía con la verificación independiente, se señala explícitamente con  **Revisar con el docente** — no se asume que el apunte tiene siempre razón, ni que este documento la tiene: ante cualquier discrepancia, el criterio de desempate es tu profesor.

---

## Preguntas teóricas

### (a) ¿Por qué un único bit de paridad detecta errores pero no corrige?

Un código de  $k$  bits sin protección tiene distancia mínima 1 (dos códigos válidos cualesquiera pueden diferir en 1 solo bit). Al agregar un bit de paridad, la distancia mínima del código pasa a ser 2 (dos palabras válidas cualesquiera —dato+paridad correcta— difieren en al menos 2 bits). Con distancia  $d=2$ : se puede **detectar** hasta  $e=d-1=1$  error, pero para **corregir** se necesita  $d \geq 2e+1$ , es decir  $e \leq (d-1)/2 = 0.5 \rightarrow$  no alcanza para corregir ni 1 error. Al fallar un bit, el chequeo de paridad indica “hay un error” pero no indica *cuál* bit falló, porque cualquiera de los  $k+1$  bits pudo haber sido el que cambió y todos producen el mismo síntoma (paridad incorrecta).

*Ejemplo:* mensaje `1101`, paridad par  $\Rightarrow P=1 \Rightarrow$  transmitido `11011`. Si se corrompe cualquiera de los 5 bits, la paridad recalculada siempre da “error”, pero no hay forma de saber cuál bit cambió con esta sola información.

### (b) ¿Por qué los códigos de Hamming no son adecuados para transmisiones?

Hamming asume una tasa de error muy baja y errores de bit **independientes entre sí** (adecuado para fallas de memoria/almacenamiento). En un canal de transmisión real el ruido suele venir en **ráfagas** (errores correlacionados, varios bits consecutivos fallan juntos), violando la hipótesis de independencia — Hamming (diseñado para 1 error aislado) pierde efectividad ante ráfagas de 2+ bits.

### (c) Tabla de codificación adaptada a español (ejemplo de respuesta abierta)

- Agregar: ñ, á, é, í, ó, ú, ¿, ¡ (uso frecuente en español, ausentes en ASCII de 7 bits original).

- Podrían eliminarse o reubicarse símbolos de uso casi nulo en texto plano en español (ej. `~`, ```), aunque hay que tener cuidado si el sistema también debe soportar programación (esos símbolos son sintaxis de varios lenguajes).

#### (d) Fin de string

- **C/C++:** carácter nulo `'\0'` (byte `0x00`) al final de la cadena.
- **Pascal (y variantes tipo “string” con longitud):** no usa terminador — se guarda la **longitud** como prefijo (primer byte o palabra) y luego los caracteres.

#### (e) ¿Qué representaciones de enteros con signo tienen distinta cantidad de positivos que negativos?

- **Valor absoluto y signo, Complemento a 1:** tienen **doble representación del cero** (+0 y -0)  $\Rightarrow$  si se cuenta el cero como “no negativo”, hay tantos códigos positivos como negativos (el desbalance está en que hay dos ceros, no en positivos vs. negativos puros).
- **Complemento a 2:** tiene **una sola representación del cero**, y por eso el rango es asimétrico:  $[-2^{n-1}, 2^{n-1}-1]$   $\rightarrow$  hay **un negativo más** que positivos.
- **Desplazamiento:** el balance depende del valor de desplazamiento  $d$  elegido; con el desplazamiento estándar  $d=2^{n-1}$  se comporta igual que complemento a 2 (un negativo de más).

#### (f) ¿Por qué IEEE 754 usa desplazamiento (exceso) para el exponente?

Porque así el patrón de bits del exponente queda en orden **monótono creciente** con su valor real (sin necesitar decodificar signo aparte). Esto permite **comparar dos floats normalizados del mismo signo usando comparación de enteros sin signo directamente** sobre toda la palabra IEEE 754, sin lógica especial.

#### (g) Ventajas y desventajas de los números desnormalizados

- **Ventaja:** permiten un **underflow gradual** — en vez de saltar abruptamente de “el número normalizado más chico” a 0, se rellena ese hueco con números cada vez más cercanos a 0, reduciendo el error relativo de redondeo cerca de cero.
- **Desventaja:** tienen **menos precisión** (el bit implícito ya no es 1, se “gasta” mantisa en ceros de relleno) y requieren manejo especial en hardware/software (suelen ser más lentos de procesar que los normalizados).

#### (h) Cotas de error en simple precisión (23 bits de mantisa)

El error máximo de redondeo (mitad del ULP) para un número normalizado  $1.f \times 2^e$  es:

cota de error =  $2^{e-24}$

Para cada rango, tomo el exponente e correspondiente a esa potencia de 2:

| Rango              | Exponente e                                     | Cota de error  |
|--------------------|-------------------------------------------------|----------------|
| [0,1]              | e=0 (en x=1; en (0.5,1) sería e=-1, cota menor) | $\leq 2^{-24}$ |
| [2,4)              | e=1                                             | $\leq 2^{-23}$ |
| [128,256)          | e=7                                             | $\leq 2^{-17}$ |
| $[2^{20}, 2^{21})$ | e=20                                            | $\leq 2^{-4}$  |

**Conclusión clave:** el error absoluto **crece** con la magnitud del número (siempre  $2^{-24}$  del valor, en términos relativos), no es constante — esto es lo que hay que remarcar en la respuesta.

## Metodología general — Enteros con signo

Para cualquier ejercicio de “representar N en formato X con n bits”:

1. Convertí el **valor absoluto** de N a binario.
2. Rellená con ceros a la izquierda hasta completar los bits de magnitud disponibles (n-1 bits si hay bit de signo separado).
3. Aplicá la transformación según el formato:
  - **Valor y signo (VA+S):** bit de signo (0=positivo, 1=negativo) + magnitud tal cual.
  - **Complemento a 1 (C1):** si es positivo, igual que VA+S. Si es negativo, invertí **todos** los bits (incluido el de signo) de la representación positiva.
  - **Complemento a 2 (C2):** C1 + 1.
  - **Desplazamiento (exceso-d):** codificás N+d como número **sin signo** de n bits (con d =  $2^{n-1}$  es el convenio estándar).

**Truco de verificación (muy útil bajo corrección de tolerancia cero):** con  $d=2^{n-1}$ , **Desplazamiento(N) = Complemento a 2(N)** con el bit de signo invertido. Si tus dos resultados no cumplen esto, revisá — es una forma rápida de auto-chequear sin rehacer toda la cuenta.

## Ejercicio 1 — Código de Hamming, capacidad de corrección

### Parte 1 — ¿corrigen errores?

| Código                      | Distancia              | ¿Corrige 1 bit?                                                                                                               |
|-----------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Hamming                     | $d=3$                  | Sí ( $e=1 \leq (d-1)/2=1$ )                                                                                                   |
| Paridad simple              | $d=2$                  | No, solo detecta                                                                                                              |
| Entrelazado 2 de 5          | $d=2$                  | No, solo detecta (todas las palabras válidas tienen exactamente dos 1's; un error cambia esa cantidad, pero no dice cuál bit) |
| Paridad horizontal+vertical | $d=4$ (en la práctica) | Sí, hasta 1 bit (la intersección de la fila y columna con paridad fallida localiza el bit)                                    |

### Parte 2 — Codificar “1001” en Hamming

**Paso 1 — ¿Cuántos bits de redundancia  $p$  necesito?** Para  $k=4$  bits de datos:  $2^p \geq p+k+1 = p+5$ .

- $p=2$ :  $4 \geq 7 \rightarrow$  falso.
- $p=3$ :  $8 \geq 8 \rightarrow$  cierto.  $\Rightarrow$   **$p=3$** , total 7 bits.

**Paso 2 — Ubicar los bits.** Las posiciones potencia de 2 (1,2,4) son los bits de paridad  $p_1, p_2, p_3$ ; el resto son los datos, en orden de posición 7 a 1: posición7= $a_4$ , posición6= $a_3$ , posición5= $a_2$ , posición4= $p_3$ , posición3= $a_1$ , posición2= $p_2$ , posición1= $p_1$ .

Dato “1001” leído como  $a_4a_3a_2a_1$ :  $a_4=1$ ,  $a_3=0$ ,  $a_2=0$ ,  $a_1=1$ .

**Paso 3 — Calcular cada  $p_i$**  (XOR de las posiciones cuyo número binario tiene ese bit en 1):

- $p_1$  cubre posiciones 1,3,5,7  $\rightarrow p_1 = a_4 \oplus a_2 \oplus a_1 = 1 \oplus 0 \oplus 1 = 0$
- $p_2$  cubre posiciones 2,3,6,7  $\rightarrow p_2 = a_4 \oplus a_3 \oplus a_1 = 1 \oplus 0 \oplus 1 = 0$
- $p_3$  cubre posiciones 4,5,6,7  $\rightarrow p_3 = a_4 \oplus a_3 \oplus a_2 = 1 \oplus 0 \oplus 0 = 1$

**Resultado:** posición 7 $\rightarrow$ 1 =  $a_4a_3a_2p_3a_1p_2p_1 = 1001100$

## Ejercicio 2 — Decodificar Hamming de 7 bits (BCD)

**Metodología:** para cada palabra recibida, calculá los tres síndromes y armá  $S=s_2s_1s_0$  (binario). Si  $S=0$ , no hay error. Si  $S \neq 0$ ,  $S$  indica **la posición** que falló — invertí ese bit y releé el dato.

$$s_0 = p_1 \oplus a_1 \oplus a_2 \oplus a_4 \quad s_1 = p_2 \oplus a_1 \oplus a_3 \oplus a_4 \quad s_2 = p_3 \oplus a_2 \oplus a_3 \oplus a_4$$

Orden de bits: posición  $7 \rightarrow 1 = m_4m_3m_2p_3m_1p_2p_1$ . Dígito BCD =  $m_4m_3m_2m_1$ .

**Palabra 1:** 1001000  $m_4=1, m_3=0, m_2=0, p_3=1, m_1=0, p_2=0, p_1=0$

$s_0=0 \oplus 0 \oplus 0 \oplus 1=1 \cdot s_1=0 \oplus 0 \oplus 0 \oplus 1=1 \cdot s_2=1 \oplus 0 \oplus 0 \oplus 1=0 \rightarrow S=011_2=3 \rightarrow$  falló la posición 3 ( $=m_1$ ).  
Se invierte:  $m_1: 0 \rightarrow 1$ .

Dato corregido:  $m_4m_3m_2m_1 = 1001_2 = 9$

**Palabra 2:** 0000000 Todos los bits en 0  $\rightarrow s_0=s_1=s_2=0 \rightarrow S=000 \rightarrow$  sin error.

Dato:  $0000_2 = 0$

**Palabra 3:** 1110100  $m_4=1, m_3=1, m_2=1, p_3=0, m_1=1, p_2=0, p_1=0$

$s_0=0 \oplus 1 \oplus 1 \oplus 1=1 \cdot s_1=0 \oplus 1 \oplus 1 \oplus 1=1 \cdot s_2=0 \oplus 1 \oplus 1 \oplus 1=1 \rightarrow S=111_2=7 \rightarrow$  falló posición 7 ( $=m_4$ ). Se invierte:  $m_4: 1 \rightarrow 0$ .

Dato corregido:  $m_4m_3m_2m_1 = 0111_2 = 7$

**Palabra 4:** 0001111  $m_4=0, m_3=0, m_2=0, p_3=1, m_1=1, p_2=1, p_1=1$

$s_0=1 \oplus 1 \oplus 0 \oplus 0=0 \cdot s_1=1 \oplus 1 \oplus 0 \oplus 0=0 \cdot s_2=1 \oplus 0 \oplus 0 \oplus 0=1 \rightarrow S=100_2=4 \rightarrow$  falló posición 4 ( $=p_3$ , un bit de redundancia, **no afecta el dato**).

Dato (sin cambios):  $m_4m_3m_2m_1 = 0001_2 = 1$

**Mensaje decodificado completo: 9 – 0 – 7 – 1  $\rightarrow$  “9071”**

## Ejercicio 3 — Enteros sin signo de 16 bits

**Metodología:** verificar primero que  $N \leq 2^{16}-1 = 65535$ ; si no, no es representable. Si sí, dividir sucesivamente entre 2 (Caso B) para obtener los bits.

**1023:**  $1023 < 65535 \checkmark$ .  $1023 = 2^{10} - 1 \rightarrow 10 \text{ unos.} \Rightarrow \mathbf{1023 = 0000001111111111_2}$

**45216:**  $45216 < 65535 \checkmark$ . Dividiendo sucesivamente entre 2:

|                                   |                                   |                                  |                                 |
|-----------------------------------|-----------------------------------|----------------------------------|---------------------------------|
| $45216 \div 2 = 22608 \text{ r}0$ | $22608 \div 2 = 11304 \text{ r}0$ | $11304 \div 2 = 5652 \text{ r}0$ | $5652 \div 2 = 2826 \text{ r}0$ |
| $2826 \div 2 = 1413 \text{ r}0$   | $1413 \div 2 = 706 \text{ r}1$    | $706 \div 2 = 353 \text{ r}0$    | $353 \div 2 = 176 \text{ r}1$   |
| $176 \div 2 = 88 \text{ r}0$      | $88 \div 2 = 44 \text{ r}0$       | $44 \div 2 = 22 \text{ r}0$      | $22 \div 2 = 11 \text{ r}0$     |
| $11 \div 2 = 5 \text{ r}1$        | $5 \div 2 = 2 \text{ r}1$         | $2 \div 2 = 1 \text{ r}0$        | $1 \div 2 = 0 \text{ r}1$       |

Leyendo los restos del último al primero:  **$45216 = 1011000010100000_2 = 0xB0A0$**

*Verificación:*  $0xB0A0 = 11 \times 4096 + 10 \times 16 = 45056 + 160 = 45216 \checkmark$

**⚠ Revisar con el docente:** el apunte del curso trae este número como `1011 0000 1110 0000`; mi cálculo (verificado sumando de vuelta) da `1011 0000 1010 0000` (difiere en el tercer nibble: 1010 vs. 1110). Antes del parcial, confirmá cuál es la correcta con tu docente o recalculando vos mismo con la división — es exactamente el tipo de error de “un bit” que la corrección no perdona.

**71822:**  $71822 > 65535 \rightarrow$  **no es representable** en 16 bits sin signo.

## Ejercicio 4 — Enteros con signo de 16 bits (524 y -3264)

**524 en binario:**  $524 = 512 + 8 + 4 = 2^9 + 2^3 + 2^2 \rightarrow 15 \text{ bits de magnitud: } 000001000001100$

**3264 en binario:**  $3264 = 2048 + 1024 + 128 + 64 = 2^{11} + 2^{10} + 2^7 + 2^6 \rightarrow 15 \text{ bits de magnitud: } 000110011000000$

### A) VALOR Y SIGNO (VA+S)

- $524 \rightarrow \text{sg}=0 + \text{magnitud} \rightarrow \mathbf{0000001000001100}$  (0x020C)
- $-3264 \rightarrow \text{sg}=1 + \text{magnitud} \rightarrow \mathbf{1000110011000000}$  (0x8CC0)

### B) DESPLAZAMIENTO (BIAS $D=2^{15}=32768$ , CONVENIO ESTÁNDAR)

- $524 + 32768 = 33292 \rightarrow \mathbf{1000001000001100}$
- $-3264 + 32768 = 29504 \rightarrow \mathbf{0111001101000000}$

*Chequeo con el truco de la Metodología general:* Desplazamiento(524) debe ser C2(524) con el bit de signo invertido:  $C2(524) = 0000001000001100 \rightarrow \text{invertir MSB} \rightarrow 1000001000001100$  ✓ coincide.

### C) COMPLEMENTO A 1 (C1)

- 524 (positivo)  $\rightarrow$  igual que VA+S  $\rightarrow 0000001000001100$
- -3264  $\rightarrow$  complemento de todos los bits de +3264 en 16 bits (  $0000110011000000$  )  $\rightarrow 1111001100111111$  (0xF33F)

### D) COMPLEMENTO A 2 (C2)

- 524  $\rightarrow$  igual que VA+S/C1  $\rightarrow 0000001000001100$
- -3264  $\rightarrow C1(-3264) + 1 = 1111001100111111 + 1 = 1111001101000000$  (0xF340)

*Chequeo cruzado:* Desplazamiento(-3264) =  $0111001101000000$  debe ser C2(-3264) con el MSB invertido:  $1111001101000000 \rightarrow \text{invertir MSB} \rightarrow 0111001101000000$  ✓ coincide exactamente.

## Ejercicio 5 — Orden de sumas sin overflow

**Regla clave:** sumar dos operandos de **igual signo** puede producir overflow (si el resultado se sale de  $[-2^{15}, 2^{15}-1]$ ); sumar dos operandos de **signo distinto nunca** produce overflow (el resultado siempre queda entre los dos operandos). La estrategia es **alternar signos** en cada suma parcial.

### Paso 1 — Identificar signo y valor de cada hex (MSB del primer nibble):

| Hex  | Nibble alto | Signo | Valor decimal |
|------|-------------|-------|---------------|
| 7744 | 0111        | +     | +30532        |
| 5499 | 0101        | +     | +21657        |
| 6788 | 0110        | +     | +26504        |
| AB68 | 1010        | -     | -21656        |
| 88BD | 1000        | -     | -30531        |
| 9879 | 1001        | -     | -26503        |



⚠ **Revisar con el docente:** el apunte trae  $AB68 = -22656$ ; recalculando ( $0xAB68 = 43880$  sin signo,  $43880 - 65536 = -21656$ ) y verificando que la suma total da 0003 con **-21656** (no con  $-22656$ , que rompe la cuenta:  $78693 - 79690 = -997 \neq 3$ ), el valor correcto es **-21656**. Puede ser un error de tipeo en el apunte original — igual, doble-chequealo vos.

## Paso 2 — Elegir un orden alternando signos:

$7744 + 88BD = 1 \rightarrow 1 + 5499 = 21658 \rightarrow 21658 + AB68 = 2 \rightarrow 2 + 6788 = 26506 \rightarrow$   
 $26506 + 9879 = 3 = 0003 \checkmark$

Cada paso suma operandos de **signo distinto** (positivo+negativo) — excepto el paso  $1 + 5499$  y  $2 + 6788$ , donde el primer operando (1 o 2) es tan chico que el resultado igual queda muy por debajo de 32767, sin riesgo de overflow. Este orden nunca se sale de  $[-32768, 32767]$ .

## Ejercicio 6 — Suma/resta en C2 con flags Z, N, C, V

**Metodología:**  $Z=1$  si el resultado es 0.  $N=1$  si el bit más significativo (bit15) del resultado es 1.  $C=1$  si hubo acarreo/desborde **sin signo** (más allá del bit 15).  $V=1$  si hubo desborde **con signo**: ocurre exactamente cuando **ambos operandos tienen el mismo signo y el resultado tiene signo distinto**.

### A) $0X2977 + 0X5689$

Sumando hex directo:  $7+9=16 \rightarrow 0, c1 \cdot 7+8+1=16 \rightarrow 0, c1 \cdot 9+6+1=16 \rightarrow 0, c1 \cdot 2+5+1=8 \Rightarrow \mathbf{0x8000}$

- $Z=0$ ,  $N=1$  (MSB=1),  $C=0$  (no hay acarreo más allá de 16 bits:  
 $10615 + 22153 = 32768 < 65536$ ), **V=1** (positivo+positivo=negativo  $\rightarrow$  overflow)

### B) $0XCAFE + 0XB007$

Sumando hex:  $E+7=21=0x15 \rightarrow 5, c1 \cdot F+0+1=16 \rightarrow 0, c1 \cdot A+0+1=11 \rightarrow B, c0 \cdot$

$C+B=23=0x17 \rightarrow 7, \text{carry-out}=1 \Rightarrow \mathbf{0x7B05}$ , con acarreo saliente.

- $Z=0$ ,  $N=0$  (MSB del resultado  $0x7B05$  es 0), **C=1** (hubo acarreo más allá de 16 bits:  
 $51966 + 45063 = 97029 > 65536$ )
- V=1** ⚠ — ambos operandos son negativos ( $0XCAFE$  y  $0XB007$  tienen MSB=1) y el resultado ( $0x7B05$ ) es positivo  $\rightarrow$  esto **es** overflow por definición.

⚠ **Revisar con el docente:** el apunte trae  $V=0$  para este ítem. Verificado dos veces (por signo de operandos/resultado, y por la regla “acarreo-hacia-el-bit15 XOR acarreo-saliente-del-bit15” =  $0 \oplus 1 = 1$ ), ambos métodos dan **V=1**. Antes de confiar en esto para el parcial, consultalo con tu

docente — puede ser un error del apunte.

### C) 0XF21C + 0X0DE4

Sumando hex:  $C+4=16 \rightarrow 0, c1 \cdot 1+E+1=16 \rightarrow 0, c1 \cdot 2+D+1=16 \rightarrow 0, c1 \cdot F+0+1=16 \rightarrow 0, \text{carry-out}=1$   
 $\Rightarrow$  **0x0000**

(Tiene sentido: 0xF21C=-3556 y 0x0DE4=+3556 son opuestos exactos.)

- **Z=1** (resultado 0), **N=0**, **C=1** (acarreo saliente), **V=0** (operandos de signo distinto  $\rightarrow$  nunca hay overflow)

### D) 0X5789 - 0X021F

**Resta = A + (complemento a 2 de B).** C1(0x021F)=0xFDE0, C2(0x021F)=0xFDE1.

Sumando 0x5789+0xFDE1:  $9+1=10=0xA, c0 \cdot 8+E=22=0x16 \rightarrow 6, c1 \cdot 7+D+1=21=0x15 \rightarrow 5, c1 \cdot 5+F+1=21=0x15 \rightarrow 5, \text{carry-out}=1 \Rightarrow$  **0x556A**

- **Z=0**, **N=0** (positivo), **C=1** (en resta,  $C=1$  = “no hubo préstamo/borrow”, consistente con  $22409 \geq 543$ )
- **V=0** (estamos sumando 0x5789 positivo + 0xFDE1 negativo  $\rightarrow$  signos distintos  $\rightarrow$  nunca overflow)

## Ejercicio 7 — Punto fijo (16 bits, 6 fraccionales)

**Formato:** 1 bit de signo + 9 bits de parte entera + 6 bits de parte fraccional = 16 bits (complemento a 2).

### A) MAYOR NÚMERO REPRESENTABLE

Parte entera máxima (9 bits, todos 1):  $2^9-1 = 511$  Parte fraccional máxima (6 bits, todos 1):  
 $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \frac{1}{64} = 63/64 = 0.984375$

**Mayor número: 511.984375**

### B) REPRESENTAR 5.125 Y -8.75

**Truco:** multiplicá el número por  $2^6=64$  (para “correr” el punto), convertí ese entero a C2 de 16 bits, y ese es directamente el patrón de bits (el punto binario queda implícito entre el bit 9 y el bit 10 desde la izquierda).

$5.125 \times 64 = 328$ .  $328 = 256 + 64 + 8 = 2^8 + 2^6 + 2^3 \rightarrow 328 = 0000000101001000_2$  (positivo, se usa tal cual)

$\Rightarrow 5.125 = 0000\ 0001\ 0100\ 1000$

$-8.75 \times 64 = -560$ . Primero  $560 = 512 + 32 + 16 = 2^9 + 2^5 + 2^4 \rightarrow 560 = 0000001000110000_2$ .  
Complemento a 2 (invertir + 1):

Invertir:  $1111110111001111 \rightarrow +1 \rightarrow 1111110111010000$

$\Rightarrow -8.75 = 1111\ 1101\ 1101\ 0000$

Verificación:  $0xFDD0 = 65536 - 560 = 64976$ , y  $64976 = 0xFDD0 \checkmark$ .

## Ejercicio 8 — Suma en IEEE 754 simple precisión

Sumar  $3EE00000h + 3D800000h$ .

**Paso 1 — Descomponer cada número (signo | exponente 8 bits | mantisa 23 bits):**

$3EE00000h = 0011\ 1110\ 1110\ 0000\ \dots 0000 \rightarrow \text{signo}=0$ ,  
exponente= $01111101 = 125 \rightarrow e = 125 - 127 = -2$ , mantisa= $11000\dots 0 \rightarrow \text{significando } 1.11 \times 2^{-2}$

$3D800000h = 0011\ 1101\ 1000\ 0000\dots 0000 \rightarrow \text{signo}=0$ ,  
exponente= $01111011 = 123 \rightarrow e = 123 - 127 = -4$ , mantisa= $000\dots 0 \rightarrow \text{significando } 1.0 \times 2^{-4}$

**Paso 2 — Igualar exponentes (al mayor, -2) y sumar:**

$1.0 \times 2^{-4} = 0.01 \times 2^{-2}$  (corrido 2 lugares)

$1.11 \times 2^{-2} + 0.01 \times 2^{-2} = 10.00 \times 2^{-2}$

**Paso 3 — Normalizar:**  $10.00 \times 2^{-2} = 1.000 \times 2^{-1}$

**Paso 4 — Re-empaquetar:** signo=0,  $e=-1 \rightarrow \text{sesgado}=126 = 01111110$ , mantisa= $000\dots 0$

**Resultado:**  $0\ 01111110\ 000000000000000000000000 = 3F000000h$

Verificación:  $3EE00000h = 1.75 \times 2^{-2} = 0.4375$ ;  $3D800000h = 1.0 \times 2^{-4} = 0.0625$ ; suma =  $0.5 = 1.0 \times 2^{-1}$   
 $\checkmark$  (y  $0x3F000000$  es, de hecho, la constante estándar conocida para 0.5f).

## Ejercicio 9 — Formatos de punto flotante custom

**Metodología:** normalizá el número como  $1.xxxx \times 2^e$ , sumá el bias correspondiente al ancho del exponente ( $\text{bias} = 2^{\text{bits}-1} - 1$ ), y truncá la mantisa al ancho pedido.

### A) 4 BITS EXPONENTE (BIAS=7), 20 BITS MANTISA

| Número              | Normalizado                            | e (sin sesgo) | ¿Representable? (rango exp: -7 a 8) |
|---------------------|----------------------------------------|---------------|-------------------------------------|
| $23 \times 2^{24}$  | $1.0111 \times 2^{28}$                 | 28            | <b>No</b> (28 excede el rango)      |
| $23 \times 2^{-24}$ | $1.0111 \times 2^{-20}$                | -20           | <b>No</b> (-20 excede el rango)     |
| 1049318             | $1.00000000001011100110 \times 2^{20}$ | 20            | <b>No</b> (20 excede el rango)      |
| 104891              | $1.1001100110111011 \times 2^{16}$     | 16            | <b>No</b> (16 excede el rango)      |

$$(23 = 10111_2 = 1.0111 \times 2^4)$$

### B) 8 BITS EXPONENTE (BIAS=127), 10 BITS MANTISA

| Número              | e   | Exp. sesgado (binario)            | Mantisa (10 bits, truncada)                                          |
|---------------------|-----|-----------------------------------|----------------------------------------------------------------------|
| $23 \times 2^{24}$  | 28  | $28 + 127 = 155 =$<br>$10011011$  | $0111000000$                                                         |
| $23 \times 2^{-24}$ | -20 | $-20 + 127 = 107 =$<br>$01101011$ | $0111000000$                                                         |
| 1049318             | 20  | $20 + 127 = 147 =$<br>$10010011$  | $0000000000$ (los bits significativos caen fuera de los primeros 10) |
| 104891              | 16  | $16 + 127 = 143 =$<br>$10001111$  | $1001100110$                                                         |

**Verificación de 1049318:**  $1049318 = 2^{20} + 2^9 + 2^7 + 2^6 + 2^2 + 2^1 = 1048576 + 512 + 128 + 64 + 4 + 2 = 1049318 \checkmark$

**Verificación de 104891:**  $104891 = 2^{16} + 2^{15} + 2^{12} + 2^{11} + 2^8 + 2^7 + 2^5 + 2^4 + 2^3 + 2^1 + 2^0 = 104891 \checkmark$

## Ejercicio 10 — Punto flotante 16 bits (signo+5exp+10mantisa)

Bias =  $2^{5-1} - 1 = 15$ .

**A =  $67 \times 2^{-7}$ :**  $67 = 1000011_2 = 1.000011 \times 2^6 \Rightarrow A = 1.000011 \times 2^{-1}$

- $e = -1$ , sesgado =  $14 = 01110$ , mantisa =  $0000110000$
- **A = 0 01110 0000110000**

**B =  $37 \times 2^7$ :**  $37 = 100101_2 = 1.00101 \times 2^5 \Rightarrow B = 1.00101 \times 2^{12}$

- $e = 12$ , sesgado =  $27 = 11011$ , mantisa =  $0010100000$
- **B = 0 11011 0010100000**

### SUMA A+B

Valor exacto:  $A = 67/128 = 0.5234375$ ,  $B = 37 \times 128 = 4736$ . Suma =  $4736.5234375$ .

En binario:  **$4736.5234375 = 1001010000000.1000011_2$**  (exacto), normalizado:

$1.0010100000001000011 \times 2^{12}$

Al truncar la mantisa a solo 10 bits ( $0010100000$ ), la contribución de A (que aparece recién en el bit 13 de la fracción) **desaparece completamente** — A queda “absorbido” por B, un fenómeno típico de punto flotante con poca precisión cuando los exponentes difieren mucho ( $12 - (-1) = 13$  posiciones, más que los 10 bits de mantisa disponibles).

**Resultado A+B en este formato = 0 11011 0010100000 (idéntico a B solo)**

### DIVISIÓN A/B

$$A/B = (67/37) \times 2^{-7-7} = 1.810810... \times 2^{-14}$$

Convirtiendo  $1.8108108...$  a binario (10 bits):  $1100111110$  (bit 11 de redondeo = 0, no se redondea hacia arriba).

- $e = -14$ , sesgado =  $-14 + 15 = 1 = 00001$

**A/B = 0 00001 1100111110**

Verificación de magnitud:  $1.8108 \times 2^{-14} \approx 0.0001105$ ; A/B directo =  $0.5234375/4736 \approx 0.0001105$  ✓

## Ejercicio 11 — IEEE 754 sin signo (modificación del estándar)

### 1. Extremos del formato IEEE 754 simple precisión estándar:

- **Mayor número (finito):** exponente sesgado máximo para no-infinito = 254 (255 se reserva para  $\infty/\text{NaN}$ )  $\rightarrow e=254-127=127$ . Mantisa toda en 1  $\rightarrow$  significando =  $2-2^{-23}$ .

$$\left| \begin{array}{l} \text{Máximo} = (2 - 2^{-23}) \times 2^{127} \end{array} \right.$$

- **Menor número mayor que cero (desnormalizado):** exponente sesgado=0, mantisa= `00...01`  $\rightarrow 2^{-23} \times 2^{-126} = 2^{-149}$

### 2. Dos formas de aprovechar el bit de signo liberado (32 bits totales, sin signo):

**Opción 1 — Un bit más de exponente (9 bits exp + 23 mantisa):** bias= $2^8-1=255$ .

- Mayor:  $(2-2^{-23}) \times 2^{255}$
- Menor >0:  $2^{-23} \times 2^{-254} = 2^{-277}$

**Opción 2 — Un bit más de mantisa (8 bits exp + 24 mantisa):** bias=127 (igual que antes).

- Mayor:  $(2-2^{-24}) \times 2^{127}$  (más precisión, mismo rango de exponente)
- Menor >0:  $2^{-24} \times 2^{-126} = 2^{-150}$

**Trade-off:** Opción 1 gana **rango** (números mucho más grandes/chicos), Opción 2 gana **precisión** (más dígitos significativos) sin cambiar el rango.

## Ejercicio 12 — Interpretar tiras de bits

**Tira 1:** `1000 1111 1110 1111 1100 0000 0000 0000` = 0x8FEFC000

**a) Complemento a 2:** MSB=1  $\rightarrow$  negativo. Valor sin signo = 2414854144. Con signo =  $2414854144 - 2^{32} = -1880113152$

**b) Entero sin signo:** 2414854144

**c) Decimal empaquetado de 4 bits:** nibbles = 8,F,E,F,C,0,0,0  $\rightarrow$  F, E y C no son dígitos BCD válidos (solo 0-9)  $\rightarrow$  esta tira NO es una representación BCD válida.

**d) Punto flotante simple precisión:** signo=1(-), exponente=00011111=31→e=31-127=-96, mantisa da significando≈1.873046875 →  $\approx -1.873046875 \times 2^{-96}$  (un número negativo extremadamente cercano a cero, casi en el límite de underflow)

**Tira 2:** todo ceros

**a) Complemento a 2: 0 b) Entero sin signo: 0 c) Decimal empaquetado:** todos los nibbles=0000, sí es BCD válido → 00000000 = 0 **d) Punto flotante:** exponente y mantisa ambos 0 → caso especial → **+0 (cero positivo)**, no un desnormalizado (para eso la mantisa tendría que ser distinta de cero).

## Ejercicio 13 — Rangos en 2 bytes (16 bits)

### 1) COMPARACIÓN DE RANGOS

| Formato                                     | Cómo se usan los 16 bits       | Rango                          |
|---------------------------------------------|--------------------------------|--------------------------------|
| a) Decimal empaquetado (BCD, 4 bits/dígito) | 4 dígitos BCD                  | 0 a 9999 (10000 valores)       |
| b) Desempaquetado (1 byte/dígito)           | 2 dígitos, 1 por byte completo | 0 a 99 (100 valores)           |
| c) Binario sin signo                        | 16 bits de magnitud pura       | 0 a 65535 (65536 valores)      |
| d) Binario con signo (C2)                   | 16 bits con signo              | -32768 a 32767 (65536 valores) |

**Conclusión:** el binario aprovecha muchísimo mejor el espacio (65536 combinaciones útiles) que el BCD (10000, desperdicia 6 de cada 16 combinaciones por nibble) o el desempaquetado (solo 100, porque gasta un byte entero de 256 combinaciones para codificar 1 dígito de 10 valores).

### 2) REPRESENTAR 564143415 EN DECIMAL EMPAQUETADO

564143415 tiene **9 dígitos**: 5-6-4-1-4-3-4-1-5

Cada dígito → 1 nibble BCD: 0101 0110 0100 0001 0100 0011 0100 0001 0101

9 nibbles = 36 bits, que no es múltiplo de 8 → se necesita **1 nibble de relleno** para completar bytes.

| 9 dígitos + 1 nibble de relleno = 10 nibbles = 40 bits = **5 bytes**

**Representación (con relleno inicial en 0):** 0000 0101 0110 0100 0001 0100 0011 0100 0001  
0101